

WORKING WITH FUNCTIONS

What a Function is

✓ **Dfn:** *A function is a named, independent section of code that performs a specific task and optionally returns a value to the calling program.*

✓ **Note:**

i. A function is named.

✓ *Each function has a unique name. By using that name in another part of the program, you can execute the statements contained in the function.*

✓ *This is known as **calling the function**. A function can be called from within another function.*

ii. A function is independent.

✓ *A function can perform its task without interference from or interfering with other parts of the program.*

iii. A function performs a specific task.

✓ *A task is a discrete job* that your program must perform as part of its overall operation, such as sending a line of text to a printer, sorting an array into numerical order, or calculating a cube root.

iv. A function can return a value to the calling program.

✓ *When your program calls a function, the statements it contains are executed.* If you want them to, these statements can pass information back to the calling program.

Example Function

```
1      #include <stdio.h>
2
3      int cube(int x);
4
5      int input, answer;
6
7      main()
8      {
9          printf("Enter an integer value: ");
10         scanf("%d", &input);
11         answer = cube(input);
12
13         printf("\nThe cube of %d is %d.\n", input, answer);
14
15         return 0;
16     }
17
18     /* Function: cube() - Calculates the cubed value of a variable */
19     int cube(int x)
20     {
21         int x_cubed;
22
23         x_cubed = x * x * x;
24         return x_cubed;
25     }
26
```

Program Analysis

- ✓ Line 3 contains the *function prototype*, a model for a function that will appear later in the program.
- ✓ A function's prototype contains the name of the function, a list of variables that must be passed to it, and the type of variable it returns, if any.
- ✓ Looking at line 3, you can tell that the function is named `cube`, that it requires a variable of the type `int`, and that it will return a value of type `int`.
- ✓ The variables to be passed to the function are called *arguments*, and they are enclosed in parentheses following the function's name.
- ✓ In this example, the function's argument is `int x`.
- ✓ The keyword before the name of the function indicates the type of value the function returns. In this case, a type `int` value is returned.

- ✓ **Line 11** calls the function `cube` and passes the variable `input` to it as the function's argument.
- ✓ The function's return value is assigned to the variable `answer`. Notice that both `input` and `answer` are declared on line 5 as **int variables**, in keeping with the function prototype on line 3.
- ✓ The function itself is called the *function definition*. In this case, it's called `cube` and is contained in lines 19 through 25.
- ✓ Like the prototype, the function definition has several parts. The function starts out with a function header on line 19.
- ✓ The *function header* is at the start of a function, and it gives the function's name (in this case, the name is `cube`).
- ✓ The header also gives the function's return type and describes its arguments. Note that the function header is identical to the function prototype **(minus the semicolon)**.

- ✓ The body of the function, lines 20 through 25, is enclosed in braces. The body contains statements, such as on line 23, that are executed whenever the function is called.
- ✓ Line 21 is a variable declaration that looks like the declarations you have seen before, with one difference: **it's local**. *Local variables are* declared within a function body.
- ✓ Finally, the function concludes with a return statement on line 24, which signals the end of the function.
- ✓ A return statement also passes a value back to the calling program. In this case, the value of the variable `x_cubed` is returned.

How Functions Work

- ✓ A C program doesn't execute the statements in a function until the function is called by another part of the program.
- ✓ When a function is called, the program can send the function information in the form of one or more arguments.
- ✓ **An *argument*** *is program data needed by the function to perform its task.*
- ✓ *The statements* in the function then execute, performing whatever task each was designed to do.
- ✓ When the function's statements have finished, execution passes back to the same location in the program that called the function.
- ✓ Functions can send information back to the program in the form of a return value.
- ✓ A function can be called as many times as needed, and functions can be called in any order.

Note 1:

- ✓ A **function prototype** provides the compiler with a description of a function that will be defined at a later point in the program.
- ✓ prototype includes a return type indicating the type of variable that the function will return.
- ✓ It also includes the function name, which should describe what the function does.
- ✓ The prototype also contains the variable types of the arguments (arg type) that will be passed to the function.
- ✓ Optionally, it can contain the names of the variables that will be passed. A prototype should always end with a semicolon.

Note 2:

- ✓ **A function definition** is the actual function. It contains the code that will be executed.
- ✓ The first line of a function definition, called the **function header**, should be identical to the function prototype, with the exception of the semicolon. A function header shouldn't end with a semicolon.
- ✓ Although the argument variable names were optional in the prototype, they must be included in the function header.
- ✓ Following the header is the function body, containing the statements that the function will perform.
- ✓ The function body should start with an opening bracket and end with a closing bracket.
- ✓ If the function return type is anything other than void, a return statement should be included, returning a value matching the return type.

Writing a Function

- ✓ The first step in writing a function is knowing what you want the function to do. Once you know that, the actual mechanics of writing the function aren't particularly difficult.
- ✓ As earlier mentioned, the first line of every function is the function header, which has three components, each serving a specific function.

✓ The Function Return Type

- The function return type specifies the data type that the function returns to the calling program.
- The return type can be any of C's data types: char, int, long, float, or double.
- You can also define a function that doesn't return a value by using a return type of void.

✓ The Function Name

- You can name a function anything you like, as long as you follow the rules for C variable names.
- A function name must be unique (not assigned to any other function or variable).
- It's a good idea to assign a name that reflects what the function does.

✓ The Parameter List

- Many functions use *arguments*, which are values passed to the function when it's called.
- A function needs to know what kinds of arguments to expect--the data type of each argument.
- You can pass a function any of C's data types.
- Argument type information is provided in the function header by the parameter list.
- If there is more than one parameter, each must be separated by a comma.

✓ Parameters Vs Arguments

- Sometimes confusion arises about the distinction between a parameter and an argument.
- **A *parameter*** is an entry in a function header; it serves as a "placeholder" for an argument.
- A function's parameters are fixed; they do not change during program execution.
- **An *argument*** is an actual value passed to the function by the calling program.
- *Each time a function is called*, it can be passed different arguments.
- In C, a function must be passed the same number and type of arguments each time it's called, but the argument values can be different.
- In the function, the argument is accessed by using the corresponding parameter name.

✓ **Note**

- 1.DO use a function name that describes the purpose of the function.**
- 2.DON'T pass values to a function that it doesn't need.**
- 3.DON'T try to pass fewer (or more) arguments to a function than there are parameters. The number of arguments passed must match the number of parameters.**

✓ The Function Body

- *The function body is enclosed in braces, and it immediately follows the function header. It's here that the real work is done.*
- When a function is called, execution begins at the start of the function body and terminates (returns to the calling program) when a return statement is encountered or when execution reaches the closing brace.

✓ Local Variables

- You can declare variables within the body of a function.
- Variables declared in a function are called **local variables**.
- *The term local means that the variables are private to that particular function and are distinct from other variables of the same name declared elsewhere in the program.*
- A local variable is declared like any other variable, using the same variable types and rules for names that you learned earlier.

✓ **Three rules govern the use of variables in functions:**

1. To use a variable in a function, you must declare it in the function header or the function body (except for global variables).
 2. In order for a function to obtain a value from the calling program, the value must be passed as an argument.
 3. In order for a calling program to obtain a value from a function, the value must be explicitly returned from the function.
- ✓ Keeping the function's variables separate from other program variables is one way in which functions are independent.

✓ Function Statements

- There is essentially no limitation on the statements that can be included within a function.
- The only thing you can't do inside a function is define another function.
- You can, however, use all other C statements, including loops, if statements, and assignment statements.
- You can call library functions and other user-defined functions.

✓ Returning a Value

- To return a value from a function, you use the return keyword, followed by a C expression.
- When execution reaches a return statement, the expression is evaluated, and execution passes the value back to the calling program.
- The return value of the function is the value of the expression.

✓ Passing Arguments to a Function

- To pass arguments to a function, you list them in parentheses following the function name.
- The number of arguments and the type of each argument must match the parameters in the function header and prototype.
- For example, if a function is defined to take two type int arguments, you must pass it exactly two int arguments--no more, no less--and no other type.
- If you try to pass a function an incorrect number and/or type of argument, the compiler will detect it, based on the information in the function prototype.
- If the function takes multiple arguments, the arguments listed in the function call are assigned to the function parameters in order:

✓ Recursion

- The term recursion refers to a situation in which a function calls itself either directly or indirectly.
- Indirect recursion occurs when one function calls another function that then calls the first function.
- C allows recursive functions, and they can be useful in some situations.
- You might be wondering where in your source code you should place your function definitions. For now, they should go in the same source code file as `main()` and after the end of `main()`.
- *Place your function prototypes before `main()` and your function definitions after `main()`.*